Seatline submission: SEZG583 Assignment 1

Coordinator: Omkar. Each member edits only their own rows and reflection, in their own commits.

Links

Item	Link
Repository	https://github.com/2025tm93161/seatline-scalable-services
Demo video (8-10 min)	_TODO before submission_
Report	`docs/report.pdf`

Team

Member	BITS ID	Service owned	Distributed-systems concept
Angshuman	2025TM93161	Booking (:8000)	Saga orchestration, state machine, recovery
Siva Aravindan	2025TM93154	Inventory (:8001)	Challenge 1: concurrency, atomic conditional update
Susheel Kumaraswamy S	2025TM93096	Payment (:8002)	Challenge 2: idempotency, request fingerprinting
Bandgar Omkar Sudhakar Sheetal	2025TM93075	Notification (:8003)	Challenge 3C: step-3 failure injection, saga evidence

Contribution table

Each member should be able to point to their commits on their feature branch.

Member	Code	Tests	Evidence and docs	Reviewed
Angshuman	`services/booking/`, `scripts/run.py`	`tests/test_e2e.py`, BR5, BR8	`docs/diagrams.md`, README	Inventory
Siva Aravindan	`services/inventory/`	`services/inventory/tests/`, BR1, BR2	`evidence/challenge-1-concurrency/`	Payment
Susheel Kumaraswamy S	`services/payment/`, `common/platform.py`	`tests/test_api_contract.py`, BR3, BR4, BR7	`evidence/challenge-2-idempotency/`	Notification
Omkar	`services/notification/` (mock delivery, failure simulation, BR6 trigger)	BR6 in `tests/test_business_rules.py`, `services/notification/tests/`	`evidence/challenge-3c-saga/`, `docs/openapi/`, this file	Booking

Business-rule ownership

Rule	Owner
BR1 no over-allocation, BR2 single release	Siva
BR3 decline is not a charge, BR4 single refund effect, BR7 replay/conflict	Susheel
BR5 confirmed is terminal	Angshuman
BR6 failed notification is never delivered	Omkar
BR8 step-3 failure compensates payment and hold	Angshuman + Omkar

Individual reflections

Each member writes their own (150-250 words): what you built, one hard problem, what you would change.

Angshuman I built the Booking service as the coordinator for the reservation flow and worked on `scripts/run.py` and the end-to-end tests. Booking persists its state and transition history while calling Inventory, Payment, and Notification over HTTP. The launcher supplies each service's configuration, waits for health checks, and shuts down its child processes. BR5 makes CONFIRMED terminal so a later request cannot move a successful booking backward. BR8 covers the case where notification fails after inventory has been held and payment charged: Booking records the failure, refunds the charge, releases the hold, and ends the booking as CANCELLED. The end-to-end tests exercise both successful booking and the failure/compensation path.

The hardest problem was coordinating several independently committed operations without a distributed transaction. A failure can happen after one service has already changed state, so Booking must know which steps completed and compensate them in the right order. Persisting transitions and testing notification failure made that behavior observable and repeatable.

I would extend the end-to-end suite to interrupt compensation itself, restart the affected dependency, and verify that a booking left in COMPENSATION_PENDING can recover without charging or releasing twice. That would cover recovery after a second failure, not only the initial saga failure.

Siva Aravindan I built the Inventory service's capacity and hold operations and contributed its focused tests. The reservation path decrements availability with a conditional SQL update, so it only succeeds when enough capacity remains; database constraints also prevent availability from going below zero or above capacity. The release path restores seats only when a hold moves from RESERVED to RELEASED. Per-reservation locking and a cancellation tombstone prevent duplicate releases from restoring seats twice and prevent a late reserve from reviving a released hold. The Challenge 1 evidence exercises the HTTP path with 20 parallel requests for five available seats: five return 201, fifteen return 409, and exactly five holds are recorded.

The hardest problem was handling concurrent operations and out-of-order retries without over-allocating or leaking capacity. Correctness depends on database operations and durable hold state, not just application-level checks; release can even arrive before its corresponding reservation.

I would extend the tests with repeated randomized runs that interleave reserve and release calls for the same IDs and assert the capacity and hold invariants after each run. That would broaden coverage beyond the fixed concurrency and late-reservation cases already tested.

Susheel Kumaraswamy S I built the Payment Service's charge, refund, and lookup endpoints, with PostgreSQL persistence and domain rules that keep declined payments uncharged and make compensation safe: refunds of charged payments

become REFUNDED, while an attempt to charge after cancellation is rejected. I also implemented the shared idempotency support used by the booking flow: keys are scoped to an endpoint, request bodies receive a canonical SHA-256 fingerprint, and the original status and response are stored for exact replay. Reusing a key with changed content returns 409 instead of silently reusing the earlier result.

The hardest problem was getting retries right under concurrency without creating duplicate business effects. The idempotency transaction lock serializes requests using the same key; the payment path also locks by booking ID, while the database's booking ID primary key prevents a second payment row. Challenge 2 evidence shows two synchronized identical booking requests resolving to one booking, one charge, and one consumed seat. A later replay returns the same booking response, while a changed payload conflicts.

I would change the demo's indefinite idempotency retention before production: define a documented retention window and cleanup policy, and integrate a payment provider's own idempotency key plus reconciliation. Local database protection alone cannot guarantee exactly-once effects across an external payment boundary.

Omkar I built the Notification Service: a mock delivery provider with its own `notification` database, a `POST /api/v1/deliveries` endpoint protected by an `Idempotency-Key`, and a `simulate_failure` switch that Booking uses as saga step 3. Because Notification is the last step, it is the dependency we use to prove compensation: when it fails, Payment refunds and Inventory releases the hold.

The hard problem was BR6, making a provider failure final. A failure is meaningful to the saga only if Notification and Booking agree on it. If a retry could later turn a FAILED delivery into DELIVERED, a customer would be told their booking was confirmed after we had already refunded them and released their seats. I solved this in three layers. The domain rule `can_deliver` rejects the transition. The FAILED row is written before the 502 is returned, so the record exists before Booking starts compensating. A PostgreSQL trigger blocks the transition even for a direct UPDATE. A per-booking advisory lock plus the shared idempotency layer stop concurrent retries from racing to insert two outcomes.

What I would change: notification is called synchronously over REST, so a slow provider holds up the booking. In production I would publish a `BookingConfirmed` event through a transactional outbox and let Notification consume it asynchronously, with bounded retries and a dead-letter queue, keeping the saga to the steps that truly need to be reversed.

AI assistance declaration

The README asks the team to declare AI assistance. Each member states, for their own part, which tools were used and for what, and confirms they can explain every mechanism they submitted.

Member	Declaration
Angshuman	GitHub Copilot assisted with drafting and checking the Booking service's saga and state-transition logic, the service launcher, and end-to-end tests. I reviewed the suggestions used in my work and can explain BR5, BR8, and the compensation and recovery flow.
Siva Aravindan	GitHub Copilot assisted with drafting and checking the Inventory service's capacity and hold logic, its BR1/BR2 tests, and Challenge 1 evidence. I reviewed the suggestions used in my work and can explain the conditional capacity update, safe release behavior, and concurrency results.

Member	Declaration
Susheel Kumaraswamy S	GitHub Copilot assisted with drafting this reflection and checking it against the Payment Service and Challenge 2 evidence. I reviewed the description and can explain the mechanisms submitted.
Omkar	Claude (Anthropic) assisted with planning the team split, demo speaking points, a pre-submission review of `main`, the Compose health-check/restart change, and drafting my reflection. I designed and wrote the Notification Service, BR6 enforcement and its tests, reviewed every AI-suggested change, and can explain each mechanism.

Demo video outline (8-10 minutes)

Time	Presenter	Content
0:00-1:30	Angshuman	Architecture, database-per-service, REST only, booking state machine
1:30-3:00	All	`scripts/run.py`, every `/health` and `/docs`, one confirmed booking
3:00-4:30	Siva	Challenge 1: 20 parallel holds, 5 succeed, 15 rejected, available = 0
4:30-6:00	Susheel	Challenge 2: simultaneous same key, replay header, changed payload → 409
6:00-8:00	Omkar + Angshuman	Challenge 3C: `fail_notification`, delivery `FAILED`, refund, release, `CANCELLED`, one `X-Request-ID` in all logs
8:00-9:30	All	Tests run, evidence folders, limitations

Pre-submission checklist

- [ ] All four feature branches merged to `main` with each member's own commits - [ ] `pytest tests services/*/tests` passes against `TEST_DATABASE_URL` - [ ] `scripts/challenges.py all` run on the final code, results committed under `evidence/` - [ ] `scripts/export_artifacts.py` run; `docs/openapi/*.json` and database snapshots refreshed - [ ] `docs/report.pdf` built with `scripts/build_report.py` - [ ] README followed from a fresh clone by someone other than its author - [ ] Every TODO in this file replaced; video link added